Department of Nano Science Technology
Faculty of Technology
Wayamba University of Sri Lanka

Centralized Workshops and Laboratory Allocation Platform

OOP and Software Design

Submitted to: Dr. U.S. Liyanarachchi
Module: NANO2142

Submitted by: Group 1

Date of Submission

05 Nov 2026

OOP Design

SYNCLAB is written in C++17 with Qt 6. Its object-oriented structure separates the user interface, validation rules and database access into distinct classes, which keeps each responsibility small and testable.

Table 1: Main classes

Class	Derived from	Responsibility
MainWindow	QMainWindow	Coordinates screens and user actions; owns the database manager and current record
IDatabaseManager	– (abstract)	Persistence contract: addRecord, updateRecord, deleteRecord, createTable, isTimeSlotAvailable, getLastError
SqliteDatabaseManager	IDatabaseManager	SQLite implementation with prepared statements and transactions
StudentRecord	–	Value object holding identity, schedule, project, key and status data
AllocationRules	–	Static validation and preparation of new and updated allocations
CustomCalendarWidget	QCalendarWidget	Draws A / P / R status counts on calendar dates
Custom dialogs	QDialog	CustomAlertDialog, CustomInputDialog, AdminSettingsDialog, SystemSettingsDialog

1. Encapsulation

Encapsulation hides internal data and exposes only controlled operations.

- **StudentRecord** keeps all fields private (identity, schedule, supervisor, project, allocation key and status) and provides getters and setters. Other classes cannot change the data directly.
- **SqliteDatabaseManager** keeps the QSqlDatabase connection, connection name and last error private. Validation, transaction start/commit and child-slot replacement are private helpers (validateRecord, beginWrite, saveRelatedFacts, replaceSlots).

Why: the window never touches SQL or connection details, so the database code can change without affecting the interface, and invalid data cannot bypass the validation inside the class.

2. Inheritance

Inheritance lets a class reuse and extend the behavior of a base class.

- **SqliteDatabaseManager** derives from the abstract class **IDatabaseManager** and implements its persistence contract.
- **MainWindow** derives from QMainWindow, **CustomCalendarWidget** from QCalendarWidget, and the prompt and settings dialogs from QDialog, reusing Qt's widget lifecycle, input and event handling.

Why: a shared interface defines what the application needs from storage, while Qt base classes avoid rewriting standard window and dialog behavior.

3. Polymorphism

Polymorphism allows the same call to run different code depending on the actual object.

- **Virtual dispatch:** MainWindow stores an IDatabaseManager pointer and constructs a SqliteDatabaseManager. Calls such as addRecord(), updateRecord() and isTimeSlotAvailable() are dispatched at run time through virtual methods. The interface has a virtual destructor so the derived object is deleted correctly.
- **Method overriding:** Qt calls CustomCalendarWidget::paintCell() through its override when drawing each date, adding the status-count decoration.

Why: MainWindow depends on the interface rather than on SQLite, so another storage implementation could be substituted without changing the window code.

4. Composition, Rules and Events

- **Composition:** MainWindow contains the current StudentRecord and owns the database manager, which closes its connection in its destructor. Qt parent–child ownership manages widgets.
- **Separate rules:** AllocationRules finds the first input error, creates the four-digit key and timestamp for a new allocation and resets student edits to Pending, so these decisions are testable without opening every screen.
- **Event-driven design:** Qt signals and slots connect button clicks, calendar selection, status changes and settings changes to application behavior.

- **Design pattern (optional):** the database interface works like a repository boundary. The code does not use a full dependency-injection framework or a database factory.

Limitation and future refactor

MainWindow still combines navigation and orchestration. A future version could extract booking and admin controllers and inject the database interface while keeping the current virtual contract.

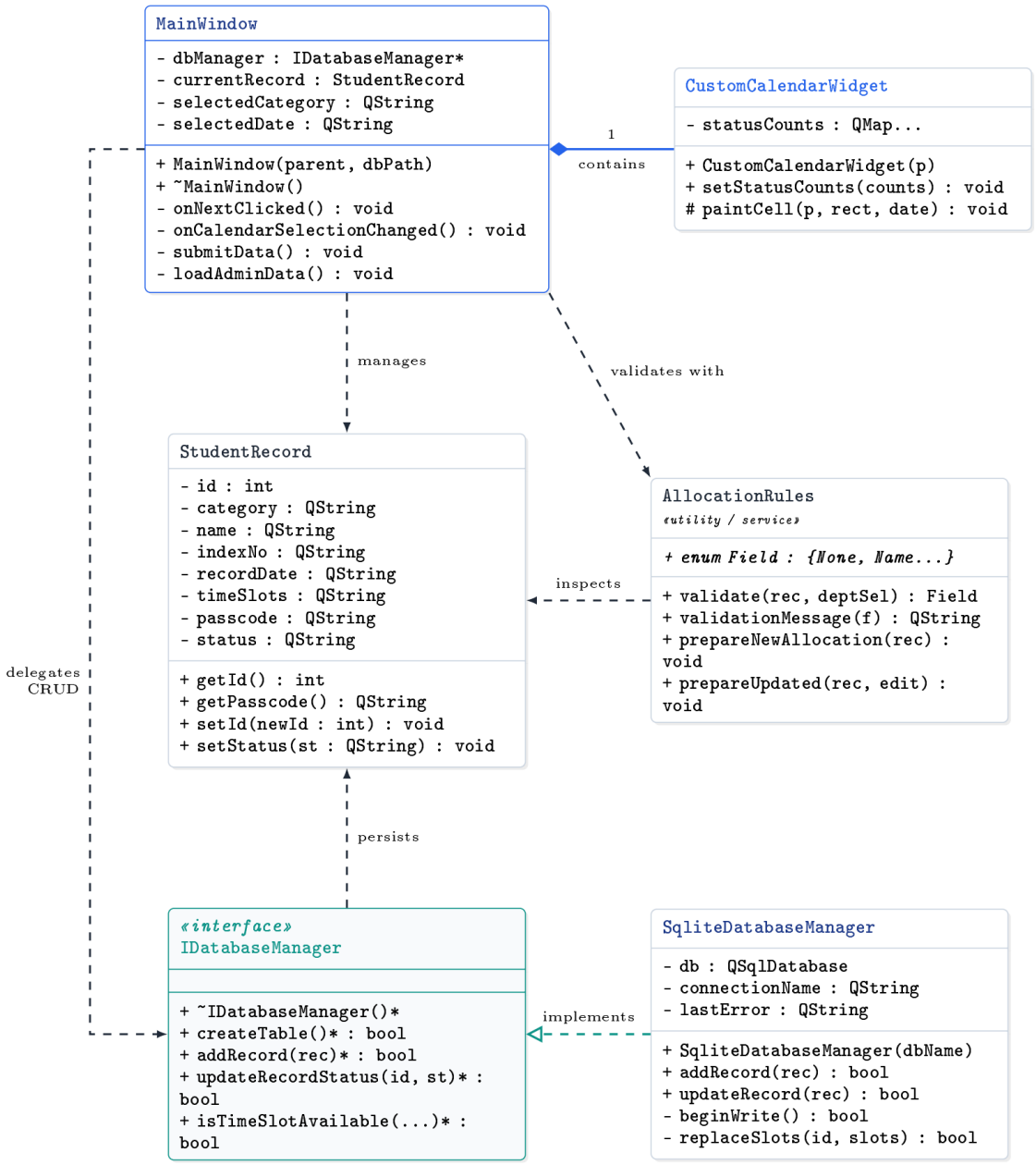


Figure 1: Class diagram of SYNCLAB